

```

// Fetches publicly-tagged surveillance
nodes from OpenStreetMap's Overpass
// API for a given bounding box, and caches
them locally via db.js.
//
// PRIVACY NOTE: the only thing sent over
the network here is a bounding box
// of map coordinates the user is currently
viewing – never the user's own
// GPS fix, never a device identifier,
never a history of movement.

import { OVERPASS_ENDPOINT } from
'../utils/constants';
import { upsertCameraNodes,
getCameraNodesInBounds } from './db';

function buildOverpassQuery({ minLat,
minLon, maxLat, maxLon }) {
  const bbox =
`${minLat},${minLon},${maxLat},${maxLon}`;
  return `
    [out:json][timeout:25];
    (
      node["man_made"]="surveillance"
    (${bbox});
      node["surveillance:type"](${bbox});
    );
    out body;
  `;

```

```

}

function parseOverpassElements(elements) {
  return elements
    .filter((el) => el.type === 'node')
    .map((el) => ({
      osmId: el.id,
      lat: el.lat,
      lon: el.lon,
      surveillanceType: el.tags?.
['surveillance:type'] ?? el.tags?.
['man_made'] ?? 'unknown',
      direction: el.tags?.
['camera:direction'] ?? null,
    }));
}

// Fetches fresh nodes for the given bbox
and merges them into the local cache.
// Throws on network failure – caller
should fall back to cached data.
export async function
syncCameraNodesForBounds(bounds) {
  const query = buildOverpassQuery(bounds);
  const response = await
fetch(OVERPASS_ENDPOINT, {
  method: 'POST',
  headers: { 'Content-Type': 'text/plain'
},
  body: query,
});

  if (!response.ok) {
    throw new Error(`Overpass request

```

```
failed: ${response.status}`);
    }

    const json = await response.json();
    const nodes =
    parseOverpassElements(json.elements ?? []);
    await upsertCameraNodes(nodes);
    return nodes;
}

// Offline-first accessor: always returns
// whatever is cached immediately;
// callers that also want a fresh sync
// should call syncCameraNodesForBounds
// separately (e.g. on viewport idle) and
// re-read the cache after.
export async function
getCachedCameraNodes(bounds) {
    return getCameraNodesInBounds(bounds);
}
```